

1. SQL

Вопрос: Приведите пример запроса с оконной функцией или многоуровневой вложенной выборкой. Как вы проверяли его производительность?

Ответ:

Использовал запросы с оконными функциями при переносе legacy-данных в новый сервис. Проверяли корректность и производительность запросов по времени выполнения и нагрузке. Смотрели, насколько долго выполняется запрос и насколько сильно грузит систему.

2. API + TLS

Вопрос: Что такое keystore? С какой проблемой при TLS/SSL сталкивались при тестировании API? Как настраивали Wiremock для SOAP?

Ответ:

С TLS/SSL работал при тестировании API. Сталкивался с проблемами некорректных ответов API: неправильные коды ошибок, неполные данные в ответе, ограничения по методам.

С SOAP работал на первом проекте: тестировали SOAP API, проверяли XSD-схемы, открывали WSDL. Wiremock самостоятельно не настраивал.

Keystore - хранение данных.

3. Логи, мониторинг и алерты

Вопрос: Из каких компонентов состоит ваш стек сбора логов? Что такое Fluentd/Filebeat? Какой дашборд в Grafana строите первым?

Ответ:

Логи смотрели через Kibana. Также использовали OpenSearch, WAN и Uber (вряд ли это, но я как-то сама не разобрала...). Grafana использовалась на первом проекте, в основном для просмотра графиков и таймингов.

Например, анализировали падения по времени — видели, что ошибки возникают в определённые часы из-за высокой нагрузки, после чего уже заходили в Kibana и искали проблему подробнее.

Fluentd и Filebeat подробно не использовал.

4. Kafka / RabbitMQ / MongoDB / Redis

Вопрос: С какими технологиями работали? Приведите пример producer/consumer сценария.

Ответ:

Работал с Kafka, RabbitMQ и MongoDB. Redis не использовал.

MongoDB использовали в основном для хранения файлов, которые приходили от внешних систем.

Kafka и RabbitMQ использовали как брокеры сообщений. Типовой сценарий:

- внешний сервис отправляет запрос;
- сервис получает данные из БД;
- данные кладутся в очередь;
- consumer вычитывает сообщение и обрабатывает его дальше.

На своей стороне выступали producer. Работали с топиками и очередями.

5. Тест-дизайн

Вопрос: Назовите 3 техники тест-дизайна, которые вы использовали.

Ответ:

Использовал:

- граничные значения;
- таблицы принятия решений;
- позитивные и негативные сценарии (happy path и негативные проверки).

Для smoke и регресса в первую очередь брали самые критичные и часто используемые кейсы проекта.

6. Kubernetes

Вопрос: Как тестировали микросервисы в Kubernetes?

Ответ:

С Kubernetes взаимодействовал через UI-сервисы и WAN.

Могли перезапускать поды, если сервис не поднялся, либо временно отключать сервис для тестирования сценариев.

Также меняли конфигурацию для тестов — например, увеличивали время отбивки сообщений в очередь.

7. Интеграционное тестирование

Вопрос: Приведите пример интеграционного теста.

Ответ:

Расскажите пример интеграционного теста, который вы проводили. Какие сервисы взаимодействовали, что проверяли?

Процесс выглядел так:

- сервис принимает запрос;
- идёт в БД за данными;
- публикует сообщение в очередь;
- consumer вычитывает сообщение и обрабатывает его на своей стороне.

Проверяли корректность передачи данных между системами и успешную обработку сообщения.